

Documentació Natural del Codi

Aquest document resumeix l'estat real del projecte `Tarraco_Fest` i les millores implementades. Última actualització: 2026-03-04.

1) Flux general de l'app

- `LandingActivity`: porta d'entrada visual.
- `AuthActivity`: login i registre.
- `HomeActivity`: pantalla principal després d'autenticar.
- `DetailActivity`: detall complet de l'esdeveniment.
- `PerfilActivity`: gestió de dades i seguretat del compte.
- `AdminPanelActivity`, `AdminUsuariosActivity`, `AdminEventosActivity`: mòdul intern d'administració.
- `AjustesActivity`: permisos i accessos del sistema.

2) Autenticació i sessió

Arxius principals:

- `app/src/main/java/com/example/tarraco_fest/Activity/AuthActivity.java`
- `app/src/main/java/com/example/tarraco_fest/Repository/UsuarioRepository.java`
- `app/src/main/java/com/example/tarraco_fest/Data/FirebaseSchema.java`

Implementat:

- Login amb Google i amb correu/contrasenya.
- Detecció de compte existent per evitar crear duplicats.
- Suport de compte mixt (Google + contrasenya Firebase).
- Flux per afegir contrasenya a un compte creat amb Google.
- Recuperació de contrasenya (diàleg dedicat + enviament d'email de restabliment).
- Validació d'usuari bloquejat abans d'entrar.
- Selecció de compte de Google (incloent-hi casos amb diversos comptes al dispositiu).
- Sessió persistent mentre l'app segueix oberta i control de tancament de sessió des del menú.

3) Home, esdeveniments i filtres

Arxius principals:

- `app/src/main/java/com/example/tarraco_fest/Activity/HomeActivity.java`
- `app/src/main/java/com/example/tarraco_fest/Repository/EventosRepository.java`
- `app/src/main/java/com/example/tarraco_fest/Adapter/EventosAdapter.java`

Implementat:

- Fusió d'esdeveniments propis (Firestore) amb esdeveniments d'API externa.
- Memòria cau (cache) temporal de l'API per reduir crides repetides.
- Refresc intel·ligent del llistat quan hi ha canvis.
- Filtres per categoria, data, preu, franja horària, distància i zona/carrer.
- Filtre de preferits (**només preferits**) integrat amb chips i panell de filtres.
- Filtre "A prop meu" utilitzant la ubicació del dispositiu.
- Persistència de filtres a **SharedPreferences**.
- Drawer lateral amb estat de selecció sincronitzat amb la pantalla actual.
- Respecte del tema clar/fosc i ajustos visuals de contrast.

4) Preferits per usuari

Arxius principals:

- `app/src/main/java/com/example/tarraco_fest/Repository/FavoritesRepository.java`
- `app/src/main/java/com/example/tarraco_fest/Activity/DetailActivity.java`
- `app/src/main/java/com/example/tarraco_fest/Activity/HomeActivity.java`

Implementat:

- Cada usuari pot guardar/treure preferits.
- L'estat de preferit es reflecteix al detall i a la Home.
- El filtre de preferits funciona sobre dades reals de l'usuari autenticat.

5) Detall d'esdeveniment i recordatoris

Arxius principals:

- `app/src/main/java/com/example/tarraco_fest/Activity/DetailActivity.java`

- `app/src/main/java/com/example/tarraco_fest/Repository/ReminderRepository.java`
- `app/src/main/java/com/example/tarraco_fest/Reminder/ReminderScheduler.java`
- `app/src/main/java/com/example/tarraco_fest/Reminder/ReminderReceiver.java`

Implementat:

- Estat contextual del recordatori:
 - esdeveniment finalitzat;
 - l'esdeveniment comença aviat;
 - esdeveniment sense data vàlida;
 - recordatori actiu.
- Validació de rang (no permet hora passada ni posterior a l'inici de l'esdeveniment).
- Recordatoris locals amb `AlarmManager` + notificació local.
- Missatgeria d'error millorada:
 - els errors importants es mostren persistents en `Snackbar` amb l'acció `Entès`;
 - l'estat també queda visible en pantalla per no perdre el missatge.
- Integració amb Google Calendar:
 - després de guardar el recordatori local, l'app ofereix afegir l'esdeveniment al Calendar;
 - utilitza l'intent de calendari natiu;
 - fallback a calendari genèric;
 - fallback final web (`calendar.google.com`) si no hi ha cap app compatible.

6) Perfil d'usuari

Objectiu:

- Permetre a l'usuari gestionar la seva informació sense exposar detalls interns de permisos.
- Mantenir el rol com a dada interna (`admin/usuario`) no visible a la UI.

Arquitectura:

- `app/src/main/java/com/example/tarraco_fest/Activity/PerfilActivity.java`
- `app/src/main/java/com/example/tarraco_fest/Repository/UsuarioRepository.java`
- `app/src/main/java/com/example/tarraco_fest/Modelo/UsuarioPerfil.java`
- `app/src/main/java/com/example/tarraco_fest/Data/FirestoreSchema.java`

UI i navegació:

- Drawer lateral amb icona d'hamburguesa.
- Seccions:
 - Informació general.
 - Seguretat.
 - Modificar dades.
- Layout principal: `app/src/main/res/layout/activity_perfil.xml`.
- Header drawer: `app/src/main/res/layout/nav_header_perfil.xml`.
- Menu drawer: `app/src/main/res/menu/menu_perfil_drawer.xml`.

Fluxos funcionals:

- Càrrega de perfil:
 - `PerfilActivity` sol·licita dades a `UsuarioRepository.cargarPerfilActual(...)`.
 - Si `usuarios/{uid}` no existeix, es crea una base mínima.
 - Es renderitza el valor real o `No establerta` si el camp està buit.
- Desa de dades editables:
 - Validació UI (nom obligatori, telèfon vàlid).
 - Actualització a Firebase Auth (`displayName`) i Firestore (`nombreMostrado, telefono, ciudad, biografia, updatedAt`).
- Seguretat:
 - Vinculació de Google des del perfil.
 - Vinculació/addició de contrasenya amb `linkWithCredential(...)`.
 - Canvi de contrasenya per email de restabliment.

Escalabilitat:

- Camps centralitzats a `FirestoreSchema.UsuarioFields`.
- Model `UsuarioPerfil` desacobla la UI del format Firestore.
- Repositori únic per a tota la lògica de perfil, preparat per estendre camps sense trencar les Activities.

7) Mòdul admin

Objectiu:

- Gestió interna d'usuaris i esdeveniments.
- No exposar capacitats d'admin a l'usuari estàndard.

Arquitectura:

- Activity:
 - `app/src/main/java/com/example/tarraco_fest/Activity/AdminPanelActivity.java`

- `app/src/main/java/com/example/tarraco_fest/Activity/Admin UsuariosActivity.java`
- `app/src/main/java/com/example/tarraco_fest/Activity/Admin EventosActivity.java`
- Repository:
 - `app/src/main/java/com/example/tarraco_fest/Repository/AdminAccessRepository.java`
 - `app/src/main/java/com/example/tarraco_fest/Repository/AdminUsersRepository.java`
 - `app/src/main/java/com/example/tarraco_fest/Repository/AdminEventosRepository.java`
- Models:
 - `app/src/main/java/com/example/tarraco_fest/Modelo/AdminUser.java`
 - `app/src/main/java/com/example/tarraco_fest/Modelo/AdminEvento.java`
- Adapter:
 - `app/src/main/java/com/example/tarraco_fest/Adapter/AdminUsersAdapter.java`
 - `app/src/main/java/com/example/tarraco_fest/Adapter/AdminEventosAdapter.java`
- Contracte:
 - `app/src/main/java/com/example/tarraco_fest/Data/FirestoreSchema.java`

Control d'accés:

- `AdminAccessRepository.verificarAccesoAdmin(...)` valida:
 - sessió activa;
 - document `usuarios/{uid}` existent;
 - `rol == admin`;
 - `bloqueado != true`.
- Si falla, les pantalles d'admin es tanquen (`finish`) i a la Home s'oculta l'accés admin.

Gestió d'usuaris:

- Càrrega amb límit i ordre per email.
- Accions:
 - bloquejar/desbloquejar;
 - canviar rol admin/usuario;
 - eliminar només quan l'usuari està bloquejat.
- Cada canvi actualitza `updatedAt`.

Gestió d'esdeveniments:

- CRUD d'esdeveniments amb estat `activo/inactivo`.
- Botó d'eliminar només visible per a esdeveniments inactius.
- Formulari amb pickers de data/hora per evitar errors manuals.
- Imatges:
 - pujada a Firebase Storage quan hi ha una configuració vàlida;
 - fallback a `imagenBase64` a Firestore si el Storage no està disponible;
 - imatge predeterminada per categoria quan no hi ha imatge pròpia.

Bootstrap del primer admin:

- Crear un usuari normal.
- A Firestore: `usuarios/{uid}` -> `rol = "admin"` i `bloqueado = false`.
- Tancar i iniciar sessió per refrescar permisos.

Seguretat recomanada:

- Regles de Firestore per impedir l'escalada de privilegis des del client.
- Regles de Storage per permetre la pujada només a admin no bloquejat.

8) Notificacions push (Firebase Messaging)

Arxius principals:

- `app/src/main/java/com/example/tarraco_fest/Push/TarracoMessagingService.java`
- `app/src/main/java/com/example/tarraco_fest/Push/PushContract.java`
- `app/src/main/java/com/example/tarraco_fest/Repository/PushTokenRepository.java`
- `app/src/main/java/com/example/tarraco_fest/TarracoFestApp.java`

Implementat:

- Registre i sincronització del token FCM a `usuarios/{uid}`.
- Canal de notificacions push general.
- Recepció de missatges remots i obertura contextual cap a Home.
- Suport de `eventId` al payload per navegar a l'esdeveniment.
- Neteja/desvinculació del token en tancar sessió.

9) Icona, splash i nom de l'app

Arxius principals:

- `app/src/main/res/values/strings.xml` (`app_name = Tarraco Fests`)
- `app/src/main/res/drawable/ic_launcher_foreground_logo.xml`
- `app/src/main/res/drawable/ic_launcher_background_tarraco.xml`

- `app/src/main/res/drawable/ic_splash_logo.xml`
- `app/src/main/res/values/themes.xml`
- `app/src/main/res/values-night/themes.xml`

Implementat:

- Separació d'icona d'escriptori i logotip de splash.
- L'escriptori usa `logo_icono` amb fons clar.
- El splash usa `logo_app` via `windowSplashScreenAnimatedIcon`.
- Nom visible de l'app actualitzat a `Tarraco Fests`.

10) Permisos i configuració Android

Arxiu principal:

- `app/src/main/AndroidManifest.xml`

Permisos actuals:

- `INTERNET`
- `POST_NOTIFICATIONS`
- `ACCESS_FINE_LOCATION`
- `ACCESS_COARSE_LOCATION`

Extras:

- `android:enableOnBackInvokedCallback="true"`.
- `adjustResize` en pantalles amb entrades de text per millorar l'experiència amb teclat.
- Registre de `ReminderReceiver` i `TarracoMessagingService`.

11) Model de dades (FirestoreSchema)

Arxiu principal:

- `app/src/main/java/com/example/tarraco_fest/Data/FirestoreSchema.java`

Estructura central:

- Col·leccions: `eventos`, `usuarios`.
- Subcol·leccions: `recordatorios`, `favoritos`.
- Camps d'usuari per a seguretat i permisos:
 - `rol`, `bloqueado`, `eliminado`, `metodosVinculados`, `tienePassword`.
- Camps per a push:

- `fcmToken`, `fcmTokens`, `fcmTokenUpdatedAt`.
- Camps de recordatori:
 - `eventStartAt`, `remindAt`, offsets de minuts/hores.

12) Consolidació de documentació

- Aquest fitxer és la font única de documentació funcional i tècnica del projecte.
- El detall de Perfil i Admin ja està integrat aquí mateix.

13) Paquet legacy

El paquet `com.example.prueba_tarracofests` es conserva com a prototip històric i no correspon al flux principal actual.

14) Suport multilinguatge

Idiomes suportats:

- Castellà (`es`)
- Català (`ca`)
- Anglès (`en`)
- Japonès (`ja`)

Arxius de recursos:

- Base: `app/src/main/res/values/strings.xml` (castellà)
- Anglès: `app/src/main/res/values-en/strings.xml`
- Català: `app/src/main/res/values-ca/strings.xml`
- Japonès: `app/src/main/res/values-ja/strings.xml`

Configuració Android:

- `app/src/main/res/xml/locales_config.xml`
- `app/src/main/AndroidManifest.xml` amb
`android:localeConfig="@xml/locales_config"`

Traducció automàtica d'esdeveniments (admin)

- En crear/editar un esdeveniment des d'admin, el sistema genera automàticament versions de `titulo` i `descripcion` per a:
 - `es`, `ca`, `en`, `ja`
- Es desa a Firestore a:
 - `tituloI18n`
 - `descripcionI18n`

- En lectura (Home/Detail) s'aplica fallback:
 - idioma actual de l'app -> `es` -> camp base (`titulo/descripcion`)